1.Selection Sort

```
#include <stdio.h>
void selection_sort(int arr[], int n) {
    int i, j, min_idx, temp;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        if (min_idx!= i) {
            temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}

void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main() {
    int arr[] = {90,12,4,1,21};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
    printf("Original: ");
    print_array( arr, n);
    selection_sort( arr, n);
   printf("Sorted: ");
    print_array(arr, n);
    return 0;
}
```

Output:

```
Original: 90 12 4 1 21
Sorted: 1 4 12 21 90

--------------------------------
Process exited after 0.1526 seconds with return value 0
Press any key to continue . . .
```

Analysis of Selection Sort:

Trace for `[64][25][12][22][11]`:

Pass	Array state	Min found	Swap
0	[64][25][12][22][11]	11 at idx 4	64 ↔ 11
1	[11][25][12][22][64]	12 at idx 2	25 ↔ 12
2	[11][12][25][22][64]	22 at idx 3	25 ↔ 22
3	[11][12][22][25][64]	25 at idx 3	no swap

1. **Time Complexity** - Count Comparisons.

The key operation is arr[j] < arr[min_idx].
Pass i=0: n−1 comparisons

Pass i=1: n−2 comparisons

...

Pass i=n-2: 1 comparison

Pass i=n-2: 1 comparison

Total comparisons:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}$$

So **T(n)=Θ(n^2) for best, average, and worst case**. Even if array is already sorted, it still does all n(n−1)/2 comparisons.

1. **Number of Swaps**:
   At most n−1 swaps, because we swap once per pass at most.
   Best case: 0 swaps if already sorted and we check min_idx != i

Worst case: n−1 swaps

This is better than Bubble Sort which can do O(n^2) swaps.

2.Insertion Sort

```
#include <math.h>
#include <stdio.h>
void insertionSort(int arr[], int N)
{

   for (int i = 1; i < N; i++)
   {
      int key = arr[i];
      int j = i - 1;
      while (j >= 0 && arr[j] > key)
      {
         arr[j + 1] = arr[j];
         j = j - 1;
      }

      arr[j + 1] = key;
   }
}

int main()
{
   int arr[] = {21, 91, 11, 5, 6};
   int N = sizeof(arr) / sizeof(arr[0]);
   printf("Unsorted array: ");
   for (int i = 0; i < N; i++)
   {
      printf("%d ", arr[i]);
   }
   printf("\n");
   insertionSort(arr, N);
   printf("Sorted array: ");
   for (int i = 0; i < N; i++)
   {
      printf("%d ", arr[i]);
   }
   printf("\n");

   return 0;
}
```

Output:

```
Unsorted array: 21 91 11 5 6
Sorted array: 5 6 11 21 91

--------------------------------
Process exited after 0.137 seconds with return value 0
Press any key to continue . . .
```

Anlaysis of Insertion Sort:

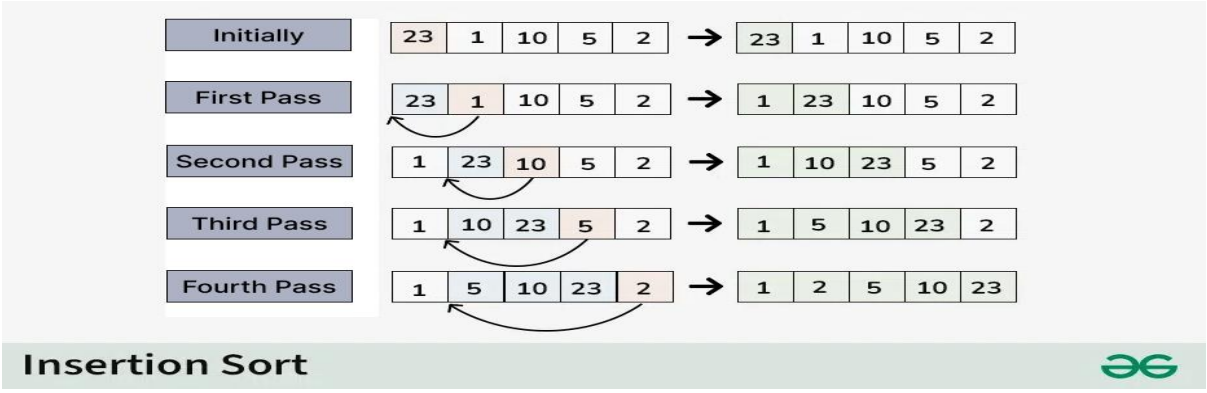
Basic operation: Comparison A[j] > key. Also shifting A [j+1] ← A[j].

Worst case: $O(n^2)$ This also means $\Theta(n^2)$ shifts. Happens when input is in descending order.

Best case: O(n) Array already sorted [1, 2,..., n]

Insertion Sort is adaptive - it runs fast on nearly sorted data.

Average case: $\Theta(n^2)$ On average, we expect each new element to be inserted halfway through the sorted portion



Case	Time Complexity	When it occurs
Best	$\Omega(n)$	Already sorted array
Average	$\Theta(n^2)$	Random array
Worst	$O(n^2)$	Reverse sorted array
Space	O(1)	In-place sorting
Stable	Yes	Equal elements keep order

3.Merge Sort

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void merge(int arr[], int left, int mid, int right) {
   int i, j, k;
   int n1 = mid - left + 1;
   int n2 = right - mid;
   int *L = (int *)malloc(n1 * sizeof(int));
   int *R = (int *)malloc(n2 * sizeof(int));
   for (i = 0; i < n1; i++) L[i] = arr[left + i];
   for (j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
   i = 0; j = 0; k = left;
   while (i < n1 && j < n2) {
      if (L[i] <= R[j])
         arr[k++] = L[i++];
      else
         arr[k++] = R[j++];
   }
   while (i < n1) arr[k++] = L[i++];
   while (j < n2) arr[k++] = R[j++];
   free(L);
   free(R);
}
void mergeSort(int arr[], int left, int right) {
   if (left < right) {
   int mid = left + (right - left) / 2;
  mergeSort(arr, left, mid);
  mergeSort(arr, mid + 1, right);
  merge(arr, left, mid, right);
   }
}
```

```
void generateRandomArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        arr[i] = rand() % 100000;
}
int main() {
    int sizes[] = {5000, 10000, 20000, 30000, 40000, 50000};
    int trials = sizeof(sizes) / sizeof(sizes[0]);
   printf("Merge Sort Timing using Divide and Conquer:\n");
    printf("=========================================\n");
    printf("Input Size\tTime Taken (seconds)\n");
    for (int t = 0; t < trials; t++) {
        int n = sizes[t];
        int *arr = (int *)malloc(n * sizeof(int));
        generateRandomArray(arr, n);
        clock_t start = clock();
        mergeSort(arr, 0, n - 1);
        clock_t end = clock();
        double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("%d\t\t%.6f\n", n, time_taken);
        free(arr);
    }
    return 0;
}
```

Output:

```
Merge Sort Timing using Divide and Conquer:
=========================================
Input Size      Time Taken (seconds)
5000            0.010000
10000           0.000000
20000           0.010000
30000           0.010000
40000           0.020000
50000           0.015000

--------------------------------
Process exited after 0.4441 seconds with return value 0
Press any key to continue . . .
```

4.DFS(Depth First Search)

```
#include<stdio.h>
int graph[10][10];
int visited[10];
int n;
void DFS(int start){
   int stack[10];
   int top = -1;
   int v, i;
   top = top + 1;
   stack[top] = start;
   while(top != -1){
      v = stack[top];
      top = top - 1;
      if(visited[v] == 0){
         printf("%d ", v);
         visited[v] = 1;
         for(i = n-1; i >= 0; i--){
            if(graph[v][i] == 1 && visited[i] == 0){
               top = top + 1;
               stack[top] = i;
            }
         }
      }
   }
}
int main(){
   int i, j, start;
```

```
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++){
        for(j = 0; j < n; j++){
            scanf("%d", &graph[i][j]);
        }
        visited[i] = 0;   // Initialize visited array
    }
    printf("Enter starting vertex: ");
    scanf("%d", &start);   // Fixed here
    printf("\nDFS Traversal: ");
    DFS(start);
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix:
1 1 0 0
1 0 0 1
1 1 1 1
1 0 1 0
Enter starting vertex: 1

DFS Traversal: 1 0 3 2
--------------------------------
Process exited after 22.97 seconds with return value 0
Press any key to continue . . .
```

4.BFS(Breadth First Search)

```
#include<stdio.h>
int graph[10][10];
int visited[10];
int n;
void BFS(int start){
   int queue[10];
   int front = 0, rear = -1;
   int v, i;
   rear = rear + 1;
   queue[rear] = start;
   visited[start] = 1;
   while(front <= rear){
      v = queue[front];
      front = front + 1;
      printf("%d ", v);
      for(i = 0; i < n; i++){
         if(graph[v][i] == 1 && visited[i] == 0){
            rear = rear + 1;
            queue[rear] = i;
            visited[i] = 1;
         }
      }
   }
}
int main(){
   int i, j, start;
   printf("Enter number of vertices: ");
```

```
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++){
        for(j = 0; j < n; j++){
            scanf("%d", &graph[i][j]);
        }
        visited[i] = 0;
    }
    printf("Enter starting vertex: ");
    scanf("%d", &start);
    printf("BFS Traversal: ");
    BFS(start);
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 1 0
1 1 1 1
0 0 1 1
Enter starting vertex: 2
BFS Traversal: 2 0 1 3
--------------------------------
Process exited after 39.8 seconds with return value 0
Press any key to continue . . .
```